```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving healthcare_dataset.csv to healthcare_dataset (1).csv
```

```python
file_name = next(iter(uploaded))
df = pd.read_csv('/content/healthcare_dataset.csv')

print("====================================")
print("FIRST 5 ROWS")
print("====================================")
print(df.head())

print("\n====================================")
print("DATASET SHAPE")
print("====================================")
print(df.shape)

print("\n====================================")
print("COLUMN NAMES")
print("====================================")
print(df.columns)

print("\n====================================")
print("MISSING VALUES")
print("====================================")
print(df.isnull().sum())

print("\n====================================")
print("DUPLICATE ROWS")
print("====================================")
print(df.duplicated().sum())
```

```
====================================
FIRST 5 ROWS
====================================
            Name  Age  Gender Blood Type Medical Condition Date of
Admission  \
0  Bobby JacksOn   30    Male         B-            Cancer        31-
01-2024
1   LesLie TErRy   62    Male         A+           Obesity        20-
08-2019
2    DaNnY sMitH   76  Female         A-           Obesity        22-
09-2022
```

```
3   andrEw waTtS   28  Female         O+          Diabetes        18-
11-2020
4  adrIENNE bEll   43  Female        AB+            Cancer        19-
09-2022

             Doctor                     Hospital Insurance Provider  \
0     Matthew Smith             Sons and Miller         Blue Cross
1   Samantha Davies                     Kim Inc           Medicare
2  Tiffany Mitchell                    Cook PLC              Aetna
3       Kevin Wells  Hernandez Rogers and Vang,           Medicare
4    Kathleen Hanna                 White-White              Aetna

   Billing Amount  Room Number Admission Type Discharge Date
Medication  \
0     18856.28131          328         Urgent     02-02-2024
Paracetamol
1     33643.32729          265      Emergency     26-08-2019
Ibuprofen
2     27955.09608          205      Emergency     07-10-2022
Aspirin
3     37909.78241          450       Elective     18-12-2020
Ibuprofen
4     14238.31781          458         Urgent     09-10-2022
Penicillin

   Test Results
0        Normal
1  Inconclusive
2        Normal
3      Abnormal
4      Abnormal

====================================
DATASET SHAPE
====================================
(55500, 15)

====================================
COLUMN NAMES
====================================
Index(['Name', 'Age', 'Gender', 'Blood Type', 'Medical Condition',
       'Date of Admission', 'Doctor', 'Hospital', 'Insurance
Provider',
       'Billing Amount', 'Room Number', 'Admission Type', 'Discharge
Date',
       'Medication', 'Test Results'],
      dtype='object')

====================================
MISSING VALUES
```

```
====================================
Name                   0
Age                    0
Gender                 0
Blood Type             0
Medical Condition      0
Date of Admission      0
Doctor                 0
Hospital               0
Insurance Provider     0
Billing Amount         0
Room Number            0
Admission Type         0
Discharge Date         0
Medication             0
Test Results           0
dtype: int64

====================================
DUPLICATE ROWS
====================================
534
```

```python
print("\n====================================")
print("DATA CLEANING STARTED")
print("====================================")
```

```
====================================
DATA CLEANING STARTED
====================================
```

```python
df = df.drop_duplicates()

numeric_cols = df.select_dtypes(include=np.number).columns
categorical_cols = df.select_dtypes(include='object').columns

for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

```
/tmp/ipykernel_26851/3154000584.py:2: FutureWarning: A value is trying
to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  df[col].fillna(df[col].median(), inplace=True)
```

```
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
/tmp/ipykernel_26851/1077209983.py:2: FutureWarning: A value is trying
to be set on a copy of a DataFrame or Series through chained
assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  df[col].fillna(df[col].mode()[0], inplace=True)
```

```
for col in categorical_cols:
    df[col] = df[col].astype(str).str.strip()

df.columns = df.columns.str.lower().str.replace(" ", "_")

print("\nData Cleaning Completed Successfully!")
```

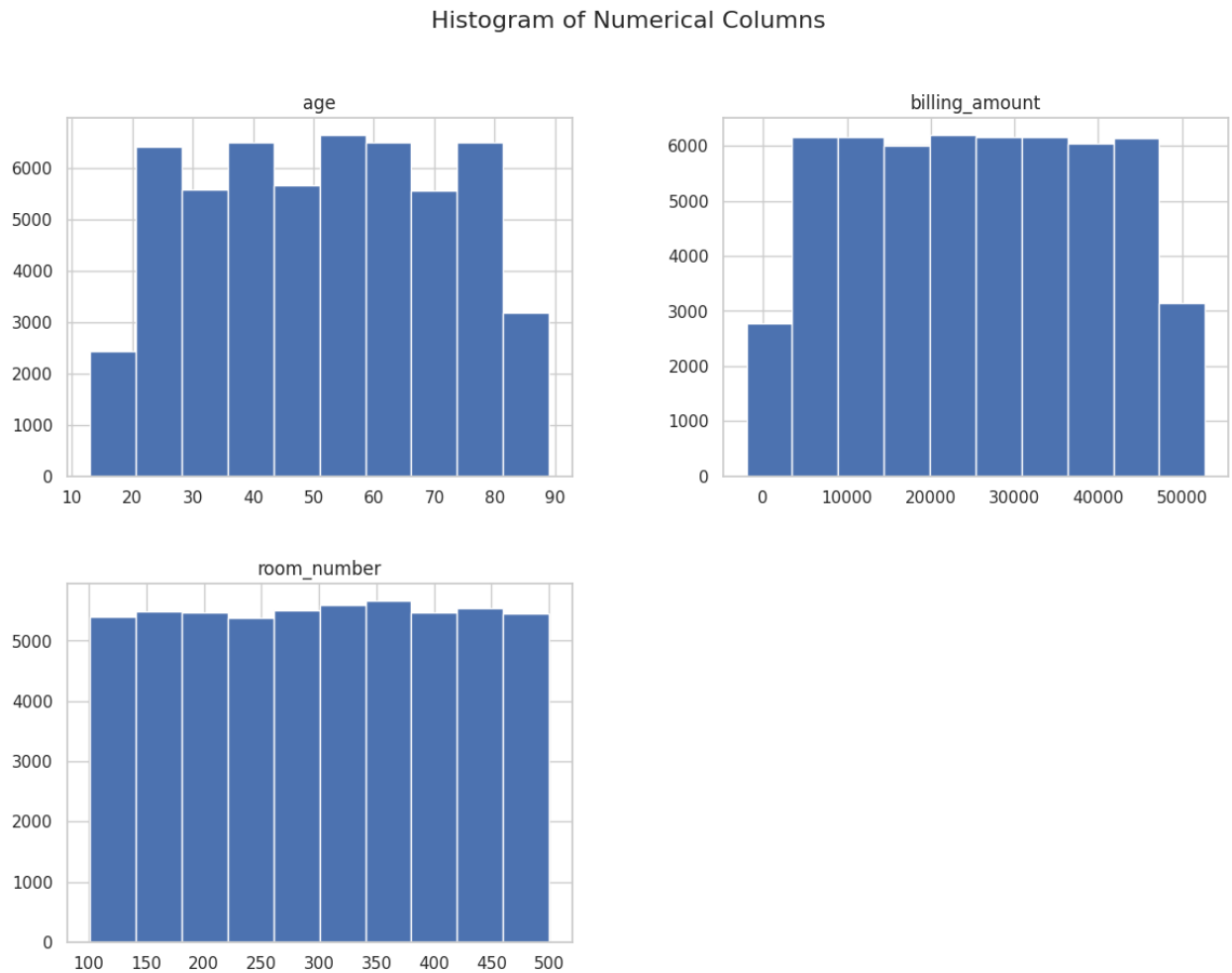
```
Data Cleaning Completed Successfully!
```

```
print("\n===================================")
print("SUMMARY STATISTICS")
print("===================================")

print(df.describe())
```

```
===================================
SUMMARY STATISTICS
===================================
                age  billing_amount   room_number
count  54966.000000    54966.000000  54966.000000
mean      51.535185    25544.306284    301.124404
std       19.605661    14208.409711    115.223143
min       13.000000    -2008.492140    101.000000
25%       35.000000    13243.718638    202.000000
50%       52.000000    25542.749145    302.000000
75%       68.000000    37819.858157    401.000000
max       89.000000    52764.276740    500.000000
```

```
sns.set(style="whitegrid")

print("\nGenerating Histograms...")

df.hist(figsize=(14,10))
plt.suptitle("Histogram of Numerical Columns", fontsize=16)
plt.show()
```

```
Generating Histograms...
```



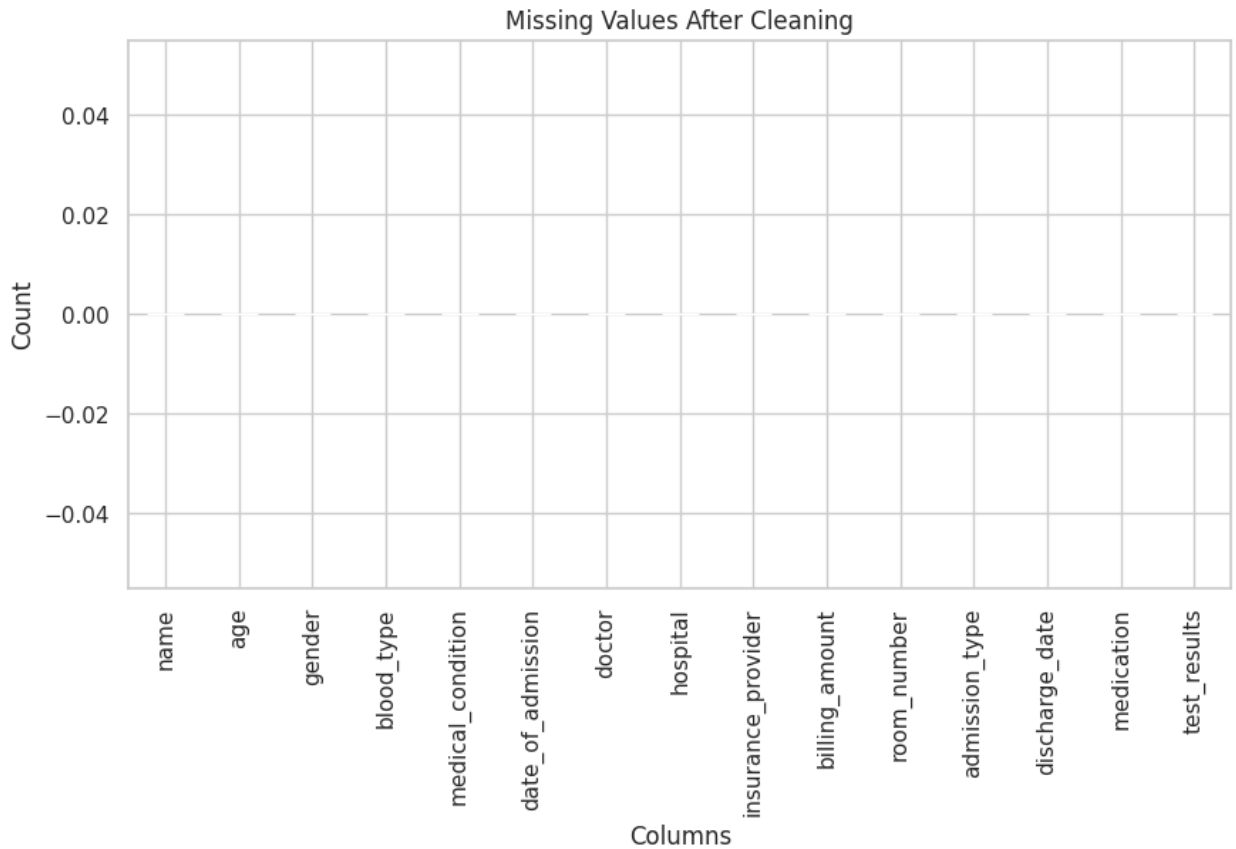

```
print("\nGenerating Correlation Heatmap...")

plt.figure(figsize=(12,6))
sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap='coolwarm',
            linewidths=0.5)
```

```
plt.title("Correlation Heatmap")
plt.show()
```

```
Generating Correlation Heatmap...
```

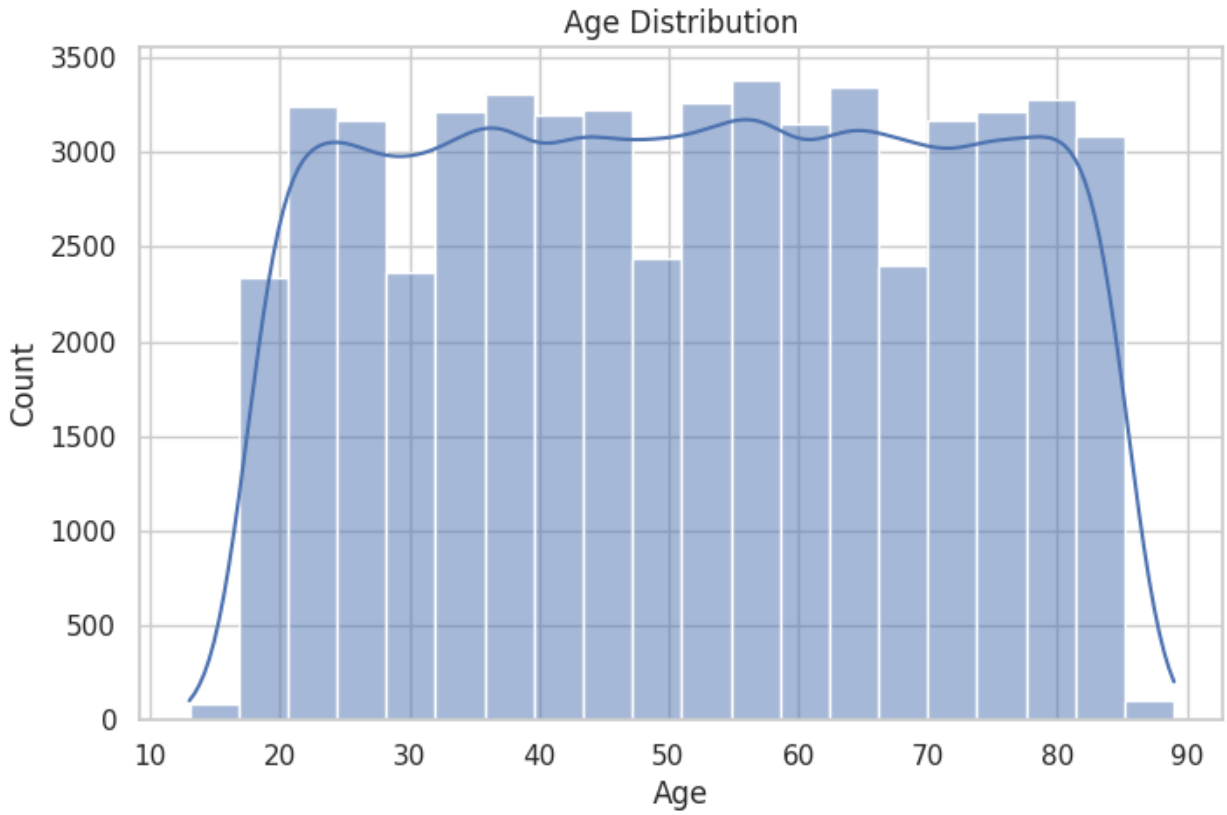
```
print("\nGenerating Missing Values Chart...")

missing = df.isnull().sum()

plt.figure(figsize=(10,5))
missing.plot(kind='bar')

plt.title("Missing Values After Cleaning")
plt.xlabel("Columns")
plt.ylabel("Count")

plt.show()
```

```
Generating Missing Values Chart...
```



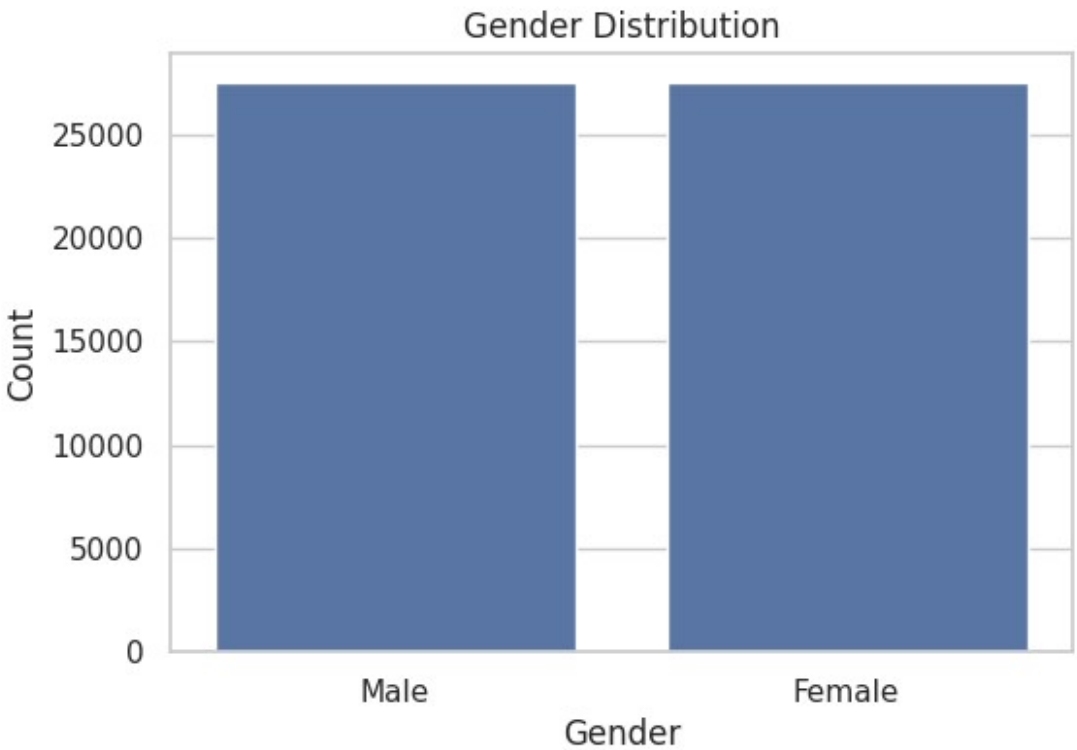

```
if 'age' in df.columns:

    print("\nGenerating Age Distribution Chart...")

    plt.figure(figsize=(8,5))

    sns.histplot(df['age'],
                 bins=20,
                 kde=True)

    plt.title("Age Distribution")
    plt.xlabel("Age")
    plt.ylabel("Count")

    plt.show()
```

```
Generating Age Distribution Chart...
```



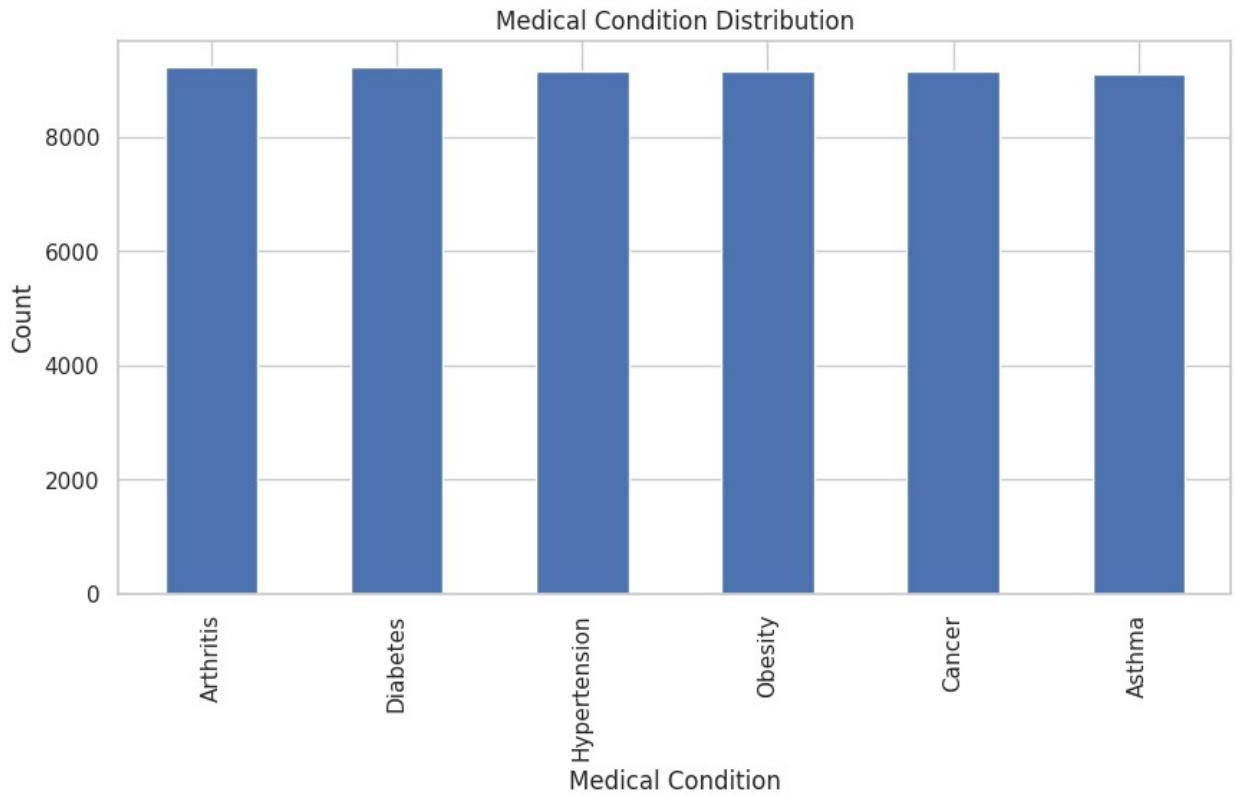

```
if 'gender' in df.columns:

    print("\nGenerating Gender Distribution Chart...")

    plt.figure(figsize=(6,4))

    sns.countplot(x=df['gender'])

    plt.title("Gender Distribution")
    plt.xlabel("Gender")
    plt.ylabel("Count")

    plt.show()
```

```
Generating Gender Distribution Chart...
```



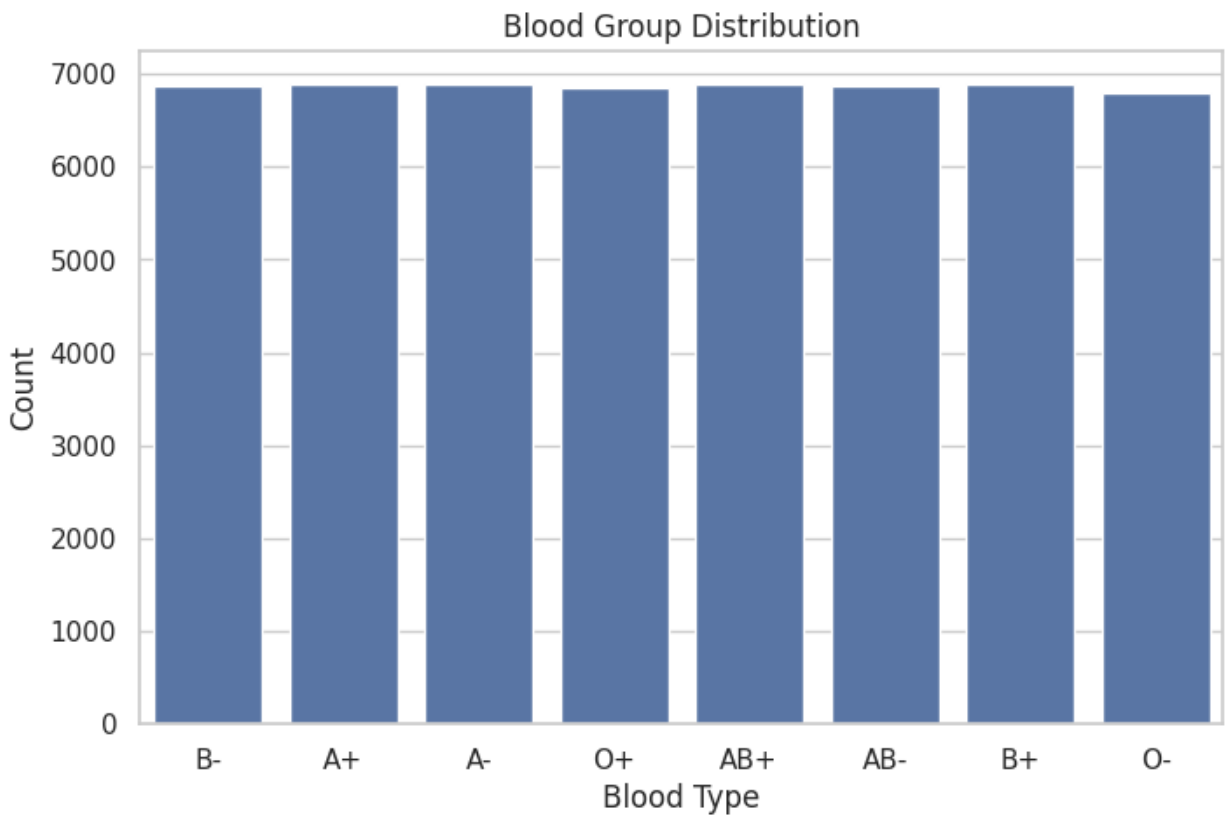

```
if 'medical_condition' in df.columns:

    print("\nGenerating Medical Condition Chart...")

    plt.figure(figsize=(10,5))

    df['medical_condition'].value_counts().plot(
        kind='bar'
    )

    plt.title("Medical Condition Distribution")
    plt.xlabel("Medical Condition")
    plt.ylabel("Count")

    plt.show()
```

```
Generating Medical Condition Chart...
```




```
if 'blood_type' in df.columns:

    print("\nGenerating Blood Type Chart...")

    plt.figure(figsize=(8,5))

    sns.countplot(x=df['blood_type'])

    plt.title("Blood Group Distribution")
    plt.xlabel("Blood Type")
    plt.ylabel("Count")

    plt.show()
```

```
Generating Blood Type Chart...
```




```
cleaned_file = "cleaned_healthcare_dataset.csv"

df.to_csv(cleaned_file, index=False)

print("\n====================================")
print("CLEANED DATASET SAVED SUCCESSFULLY")
print("====================================")

print(f"File Name: {cleaned_file}")
```

```
====================================
CLEANED DATASET SAVED SUCCESSFULLY
====================================
File Name: cleaned_healthcare_dataset.csv
```

```
files.download(cleaned_file)
```

```
<IPython.core.display.Javascript object>
```

```
<IPython.core.display.Javascript object>
```

```
print("\n====================================")
print("PROJECT COMPLETED SUCCESSFULLY!")
print("====================================")
```

```
==================================
PROJECT COMPLETED SUCCESSFULLY!
==================================
```